

CAS4160 Homework 1: Imitation Learning

[Minsung Kim]
[2020142087]

1 Introduction

Imitation Learning is a technique in Reinforcement Learning that learns a policy by utilizing expert demonstration data. While traditional reinforcement learning methods require careful design of a reward function and a large number of interactions with the environment, Imitation Learning enables more efficient acquisition of high-performing policies by imitating state-action pairs provided by the expert. This approach is particularly valuable for real-world problems where designing an effective reward function is challenging or impractical.

In this assignment, we implement and conduct a comparative analysis of two major approaches to Imitation Learning: BC (Behavioral Cloning) and DAgger (Dataset Aggregation).

2 Play with Reinforcement Learning Environments

GymTutorial.ipynb is provided as a tutorial that demonstrates how to construct Reinforcement Learning environments. By executing this tutorial, you will become familiar with the usage of Gym environments. Below are the answers to the questions in the assignment that ask whether GymTutorial.ipynb has been properly executed.

2.1 Questions:

1. Describe the role of the `step()` function. What kind of information does it return?

Answer:

The `step()` method applies the selected action to the environment, transitioning it to the subsequent state.

It returns the following elements:

- **observation**: The observation corresponding to the new state after the action is executed.
- **reward**: The immediate reward signal received from the environment based on the action taken.
- **terminated**: A boolean indicating whether the episode has reached a natural terminal state (e.g., task success or failure).
- **truncated**: A boolean indicating whether the episode was prematurely ended due to an artificial constraint, such as a maximum episode length or other external limits.
- **info**: A dictionary of supplementary information provided by the environment. In the case of Ant-v4, this includes detailed reward breakdowns (`reward_forward`, `reward_ctrl`, `reward_survive`) and kinematic information such as the ant's x-position and x-velocity.

2. Describe the role of the `reset()` function in a gym environment. What is the return value of this function?

Answer:

The `reset()` function resets the environment to its initial state.

The return value of the `reset()` function is as follows:

- The observation corresponding to the initial state of the environment.

3. How can you figure out if some gym environment has a discrete action space or continuous action space?

Answer:

The structure and properties of the action space can be inspected by printing `env.action_space`.

Typical outputs include:

- `Discrete(2)` → discrete action space (e.g., 2 possible actions)
- `Box(-1.0, 1.0, (8,), float32)` → continuous action space (e.g., 8-dimensional actions bounded between -1.0 and 1.0)

3 Behavioral Cloning

3.1 Implement Behavioral Cloning

3.1.1 BC Results

The following shows the result of implementing Behavioral Cloning and running `run_hw1.py` using the default hyperparameter settings provided in the assignment except `eval_batch_size`.

Table 1: `ep_len=1000`, `eval_batch_size=10000`, else default

Environment	Performance (Mean Return $\pm$ Std)	Expert Average
Ant-v4	4575.17 $\pm$ 74.49	4713.65
HalfCheetah-v4	3817.51 $\pm$ 113.38	4205.78
Hopper-v4	860.78 $\pm$ 246.66	3772.67
Walker2d-v4	602.29 $\pm$ 538.70	5566.85

As can be observed in [Figure 1](#), the agents performing actions in the Ant-v4 and HalfCheetah-v4 environments exhibit quadrupedal locomotion. In contrast, the agents in the Hopper and Walker2d environments are seen performing bipedal locomotion or, in some cases, standing on a single foot. This observation suggests that the early termination in Hopper and Walker2d is likely caused by instability during locomotion, which in turn leads to lower reward values.

Table 2 shows the episode lengths achieved during evaluation. This clearly indicates that episodes in the Hopper and Walker2d environments were terminated early. Both Table 1 and Table 2 report results obtained under the same evaluation settings: `ep_len=1000` and `eval_batch_size=10000`.

Table 2: Evaluation Average Episode Length Comparison

Environment	Evaluation Average Episode Length
Ant-v4	1000.0
HalfCheetah-v4	1000.0
Hopper-v4	247.4
Walker2d-v4	305.5

3.2 Rendering and Evaluating Rollouts

3.2.1 Environment Rendering

Figure:

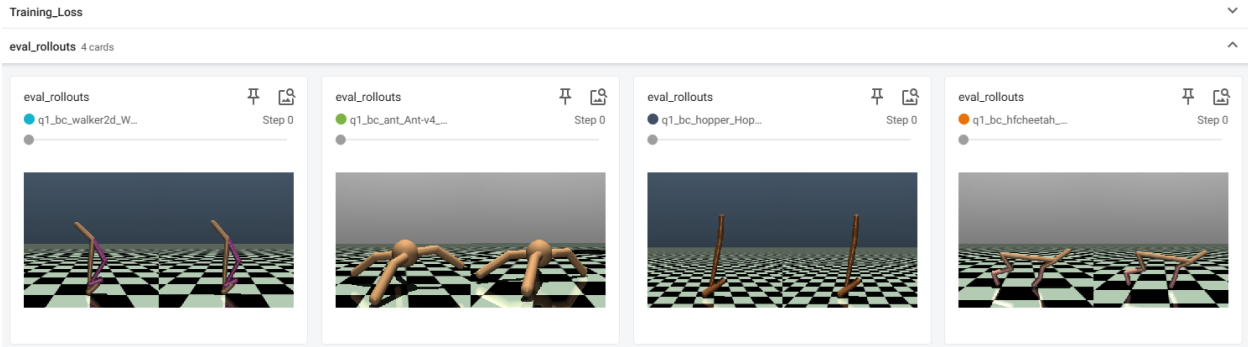


Figure 1: Rollout Results of each environment

3.3 Hyperparameter Tuning of Behavioral Cloning

3.3.1 Hyperparameter Tuning Results

Hyperparameter: To investigate the effect of the number of gradient steps per iteration, the hyperparameter `--num_agent_train_steps_per_iter` was varied across the values 1000, 2000, 3000, 4000, and 5000.

For evaluation, the following settings were used: `ep_len=1000`, `eval_batch_size=10000`, with all other parameters kept at their default values.

Plot:

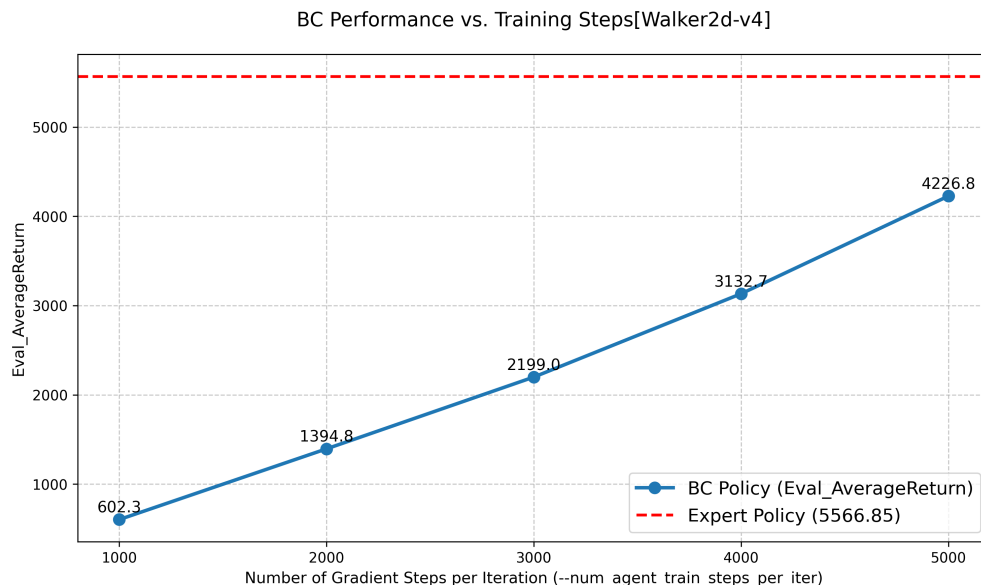


Figure 2: Performance comparison based on Walker2d-v4 environment. Network architecture: MLP with 2 hidden layers with size of 64. Training data: full expert dataset (expert_data_Walker2d-v4.pkl). Hyperparameters: num_agent_train_steps_per_iter=1000, train_batch_size=100, eval_batch_size=10000, ep_len=1000.

Rationale: If the number of gradient steps for policy training is too small, the amount of training may be insufficient, potentially leading to underfitting. Therefore, increasing the number of gradient steps allows the policy to be trained to behave closer to the ground-truth expert demonstrations.

However, when the number of steps exceeds 5000, the average return actually begins to decrease. This observation indicates that overfitting starts to occur when the number of gradient steps reaches or surpasses 5000.

4 DAgger

4.1 Compare BC and DAgger

4.1.1 DAgger Result

Similar to the Behavioral Cloning experiments, evaluation was conducted using the same fixed settings: ep_len=1000, eval_batch_size=10000, with all other hyperparameters kept at their default values.

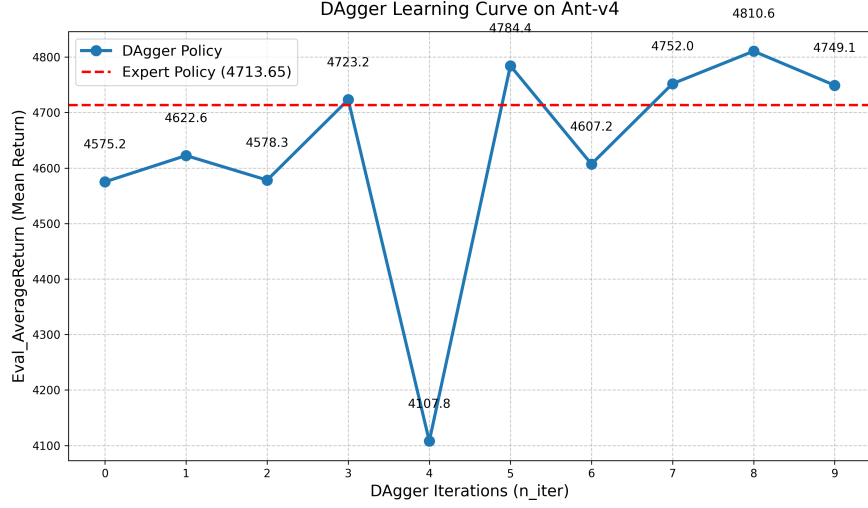


Figure 3: DAgger learning curve for Ant-v4. Network architecture: MLP with 2 hidden layers with size of 64. Training data: full expert dataset (expert_data_Ant-v4.pkl). Hyperparameters: num_agent_train_steps_per_iter=1000, train_batch_size=100, eval_batch_size=10000, ep_len=1000.

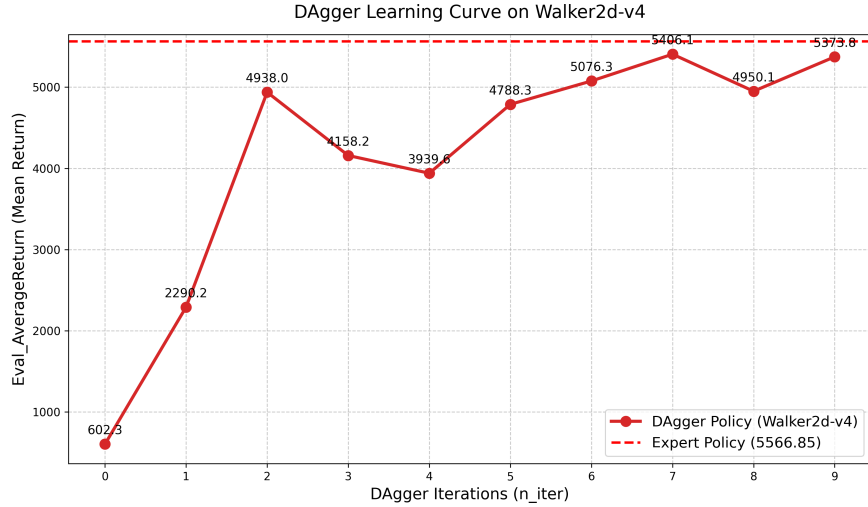


Figure 4: DAgger learning curve for Walker2d-v4. Network architecture: MLP with 2 hidden layers of size 64. Training data: full expert dataset (expert_data_Walker2d-v4.pkl). Hyperparameters: num_agent_train_steps_per_iter=1000, train_batch_size=100, eval_batch_size=10000, ep_len=1000.

4.1.2 Justification of the Result

Table 3: Mean Return Comparison (eval_batch_size=10000, else default)

Environment	Mean Return of BC	Mean Return of DAgger
Ant-v4	4575.17	4749.1
Walker2d-v4	602.29	5373.8

Based on the results presented in Table 3, we compare the performance of Behavioral Cloning, DAgger, and the expert policy, and analyze the underlying reasons for the observed differences.

All experiments were conducted under identical conditions to ensure a fair comparison: the same network architecture (two hidden layers with 64 units each), the full expert dataset, and the same set of hyperparameters (`num_agent_train_steps_per_iter=1000`, `train_batch_size=100`, `learning_rate=5e-3`, `ep_len=1000`, `eval_batch_size=10000`).

In the Ant-v4 environment, BC alone already achieved approximately 97% of the expert’s performance (4575.17), and applying DAgger further improved the average return to 4749.1, slightly surpassing the expert level. This outcome indicates that Ant-v4 is a relatively stable quadrupedal locomotion task, allowing BC to imitate the expert actions reasonably well even without additional data aggregation. The additional relabeling effect provided by DAgger was positive but relatively modest in this environment.

In contrast, for Walker2d-v4, BC achieved only about 11% of the expert performance. However, after performing 10 iterations of DAgger, the average return increased dramatically to 5373.8, reaching approximately 96.5% of the expert level. This rapid improvement is closely tied to the highly unstable nature of bipedal locomotion in Walker2d-v4. With BC, the policy is trained solely on the expert dataset; any deviation from the expert state distribution quickly leads to falling and early termination, resulting in poor performance. DAgger, on the other hand, iteratively collects new trajectories under the current policy, labels them with the expert policy, and retrains effectively supplementing the policy with experience in previously unseen states. This mechanism enables stable and substantial performance gains even in highly unstable environments.

The core mechanism of DAgger—expert relabeling of newly collected data—directly addresses the primary weakness of BC: compounding errors. In environments with high instability, such as Walker2d-v4, pure BC suffers from severe performance degradation, whereas repeated expert-guided data augmentation via DAgger progressively corrects the policy and leads to robust improvement.

Through this comparison, we confirm that DAgger consistently outperforms BC and demonstrates particularly pronounced advantages in environments with high instability, such as Walker2d-v4.